College of Engineering and Computer Science

UNIVERSITY OF CENTRAL FLORIDA

PowerInfer

Fast Large Language Model Serving with a Consumer-grade GPU

Based on Song, Mi, Xie, and Chen (SOSP 2024)

Presentation goal

Teach the core **methodology**, explain **why it works**, and show the practical impact of running large models on cheaper local hardware.

Team #7

Hai Nguyen Josh Lowe

Cuong Dang Lam Nguyen

Roadmap

Part I: Why this paper matters

- The local LLM serving bottleneck
- The locality insight behind PowerInfer
- Why offloading alone is not enough

Part II: Method + impact

- Offline hot/cold neuron placement
- Online predictors and hybrid execution
- Results, use cases, and limitations

What to listen for

PowerInfer is not just "offload to CPU." Its contribution is a **neuron-aware split**: keep the frequently active part of the model on GPU, predict which neurons matter at runtime, and let CPU handle the rare cases efficiently.

Why local LLM serving is hard

Problem setting

- Consumer GPUs have limited VRAM, but every generated token needs access to the full model.
- Quantization helps, but large models still exceed cards like the RTX 4090.
- Standard layer-wise CPU/GPU offloading reduces memory pressure but often creates **high latency**.

Why existing offloading stalls

- **PCIe bottleneck:** moving weights back and forth is slow.
- **CPU bottleneck:** if too much of the model sits on CPU, the faster GPU becomes underused.
- **Locality mismatch:** hardware likes a small hot working set in GPU memory, but naive inference touches the whole model.

Full model access every token

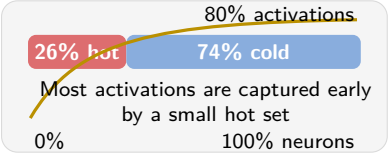
CPU-GPU transfer overhead

Low-latency local serving suffers

Key insight: neuron activations are highly skewed

Power-law activation

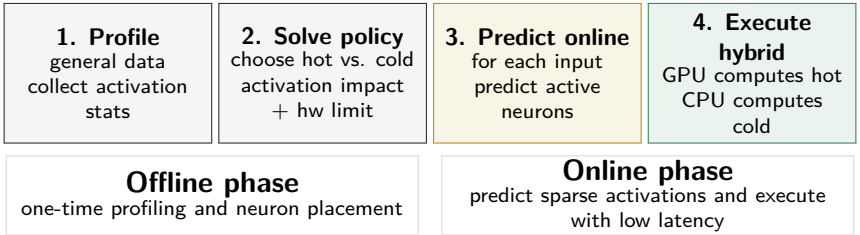
In the OPT-30B MLP blocks, about **26% of neurons account for 80% of activations**. These are the **hot neurons**. The rest are **cold neurons**: still important, but much less frequently active.



Why this matters

- Hot neurons are good candidates to preload on the **GPU**.
- Cold neurons can stay on the **CPU** and be used only when needed.
- The hot set is stable across tasks: the paper reports >**90% overlap** in the top 20% hottest neurons across several downstream tasks.

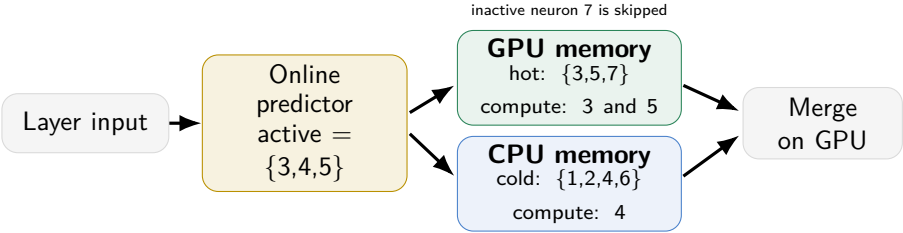
PowerInfer overview: offline policy + online hybrid execution



Main idea in one sentence

PowerInfer turns **"run the whole model everywhere"** into **"keep the frequently used neurons near the GPU, and only wake up the rare ones when prediction says they matter."**

How one layer runs inside PowerInfer



What changes vs. naive inference?

- Prediction is **input-specific**.
- Only the neurons predicted to be active are computed.
- GPU and CPU work in parallel.
- Final merging stays on the GPU, keeping the fast device on the critical path.

Method detail #1: adaptive sparsity predictors

Problem

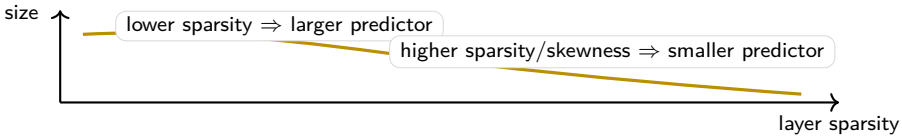
Fixed-size predictors are too expensive. The paper notes that predictors for OPT-175B would occupy about **27 GB** on GPU if designed naively.

Design

Predictor size is chosen **per layer**, based on that layer's sparsity and skewness. Hidden-layer width is tuned iteratively until perplexity stays near baseline.

Result

Predictor parameters are reduced to roughly **6%** of the LLM parameters while keeping the integrated model within **0.1% perplexity gap** from baseline.



Takeaway

Prediction is essential to exploit sparsity, but PowerInfer makes the predictor small enough that it does not eat the GPU memory it is trying to save.

Method details #2 and #3: efficient execution and placement

Neuron-aware sparse operators

- PowerInfer needs **dynamic, neuron-level sparsity**, so generic sparse libraries are a poor fit.
- It computes only the rows/columns for active neurons instead of converting full matrices at runtime.
- Result: less conversion overhead and better sparse execution on both GPU and CPU.

Small bookkeeping overhead

Neuron-index tables add only about **9 MB** even for OPT-175B.

Placement solver + hybrid scheduler

- Offline placement is cast as an **integer linear program**.
- The solver weighs activation frequency, communication, and hardware limits.
- Online, CPU/GPU executors pull ready operators from a DAG and synchronize only when needed.

Load-balancing effect

On PC-High, GPU share of neuron load rises from about **20% to 70%** on average.

Experimental setup

Hardware tiers

PC-Low	RTX 2080Ti, 11 GB, PCIe 3.0
PC-High	RTX 4090, 24 GB, PCIe 4.0
Server	A100, 80 GB

Models

OPT (7B–175B), Falcon-40B, Bamboo-7B, and LLaMA-family variants.

Workloads and baselines

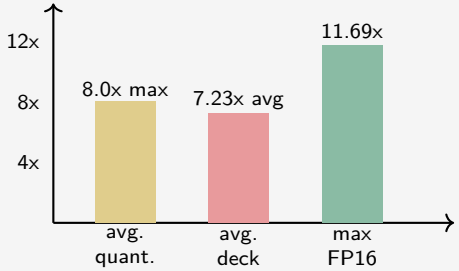
- Workloads from **Chatbot Arena**, **ChatGPT prompts**, and **Alpaca**
- Baselines: **llama.cpp** and **SpecInfer**
- Metrics: tokens/s, latency, load split, and accuracy

Important context

PowerInfer works best in the **local, low-batch regime** where latency matters more than throughput. That is exactly the setting where consumer hardware is most attractive.

Results: large speedups without losing accuracy

End-to-end performance



Practical takeaway

- On an RTX 4090, PowerInfer reaches about **82% of A100 token rate** for OPT-30B.
- Average measured speeds on RTX 4090 are **13.20 tokens/s** for quantized and **8.32 tokens/s** for non-quantized models.
- Predictor overhead is usually $<$**10%** of total inference time.

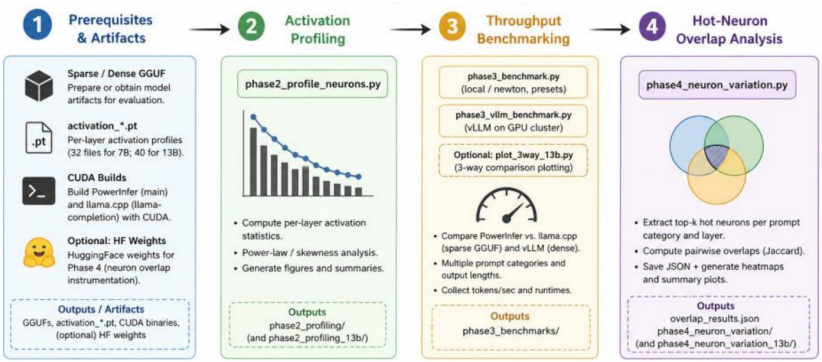
Accuracy

The paper reports that these gains come while retaining model accuracy across evaluated LLMs.

Why the gains happen

More useful work stays on the GPU, and repeated PCIe transfers are avoided.

Our Experimental Results



Shared Prompt Suite (Table tab:prompts) — Used in Phases 3 and 4

Category	Creative	Code	Factual	Reasoning	Conversational
Example Prompt (truncated)	Once upon a time in a magical kingdom, ...	Write a Python function that takes a list of ...	Explain the process of photosynthesis in ...	If all roses are flowers and some flowers ...	Hi! How are you today? Can you tell me ...

The same fixed prompts are used for throughput (Phase 3) and hot-neuron overlap (Phase 4).

Benchmark Methodology

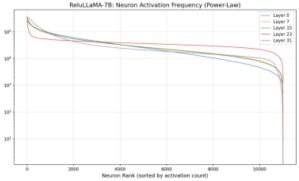
Shared prompt suite

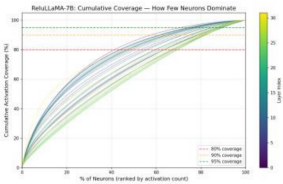
Creative	Once upon a time in a magical kingdom, there lived a young princess who dreamed of exploring the world beyond the castle walls.
Code	Write a Python function that takes a list of integers and returns the longest increasing subsequence using dynamic programming.
Factual	Explain the process of photosynthesis in plants, including the light-dependent and light-independent reactions.
Reasoning	If all roses are flowers and some flowers fade quickly, can we conclude that some roses fade quickly? Explain your reasoning step by step.
Conversational	Hey, how are you doing today? I was thinking about learning a new programming language. Any suggestions?

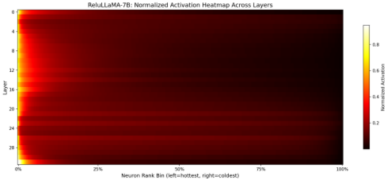
Benchmark summary

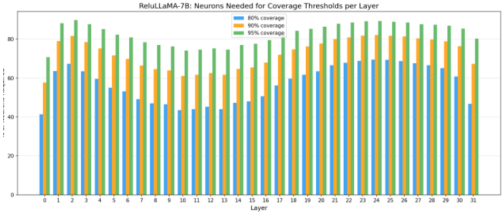
Metric	**PowerInfer (CPU)**	**llama.cpp (CPU)**	**vLLM (GPU)**
Avg generation speed	∼10.4 tok/s (high variance)	∼15.4 tok/s	∼54 tok/s
Speed range	0.77–20.59 tok/s	3.18–17.95 tok/s	24.4–54.2 tok/s
Prompt-type variance	Very high	Moderate	<1% (after warm-up)
Hardware	Apple M4 Max (CPU)	Apple M4 Max (CPU)	Newton GPU (CUDA)

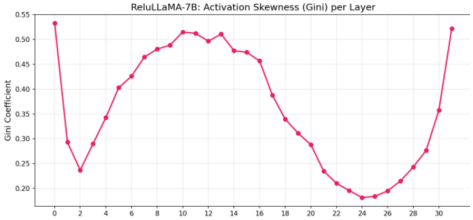
Results: ReluLLaMA-7B



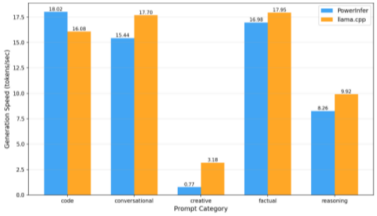


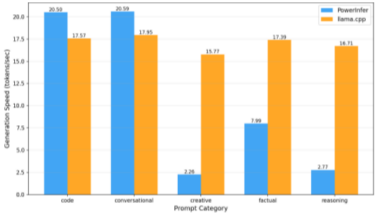


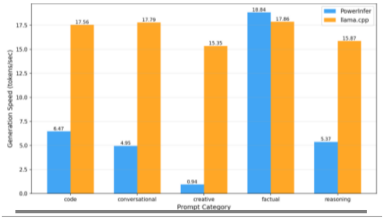




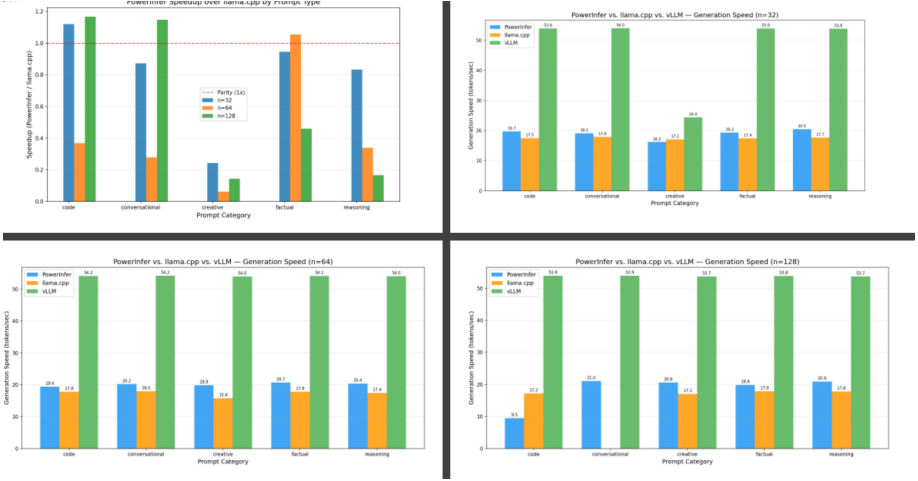
Results: ReluLLaMA-7B throughput plots



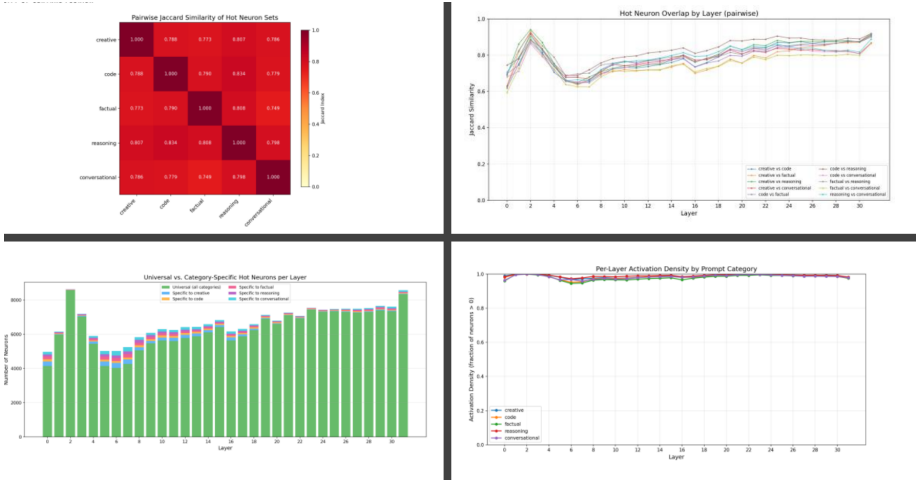




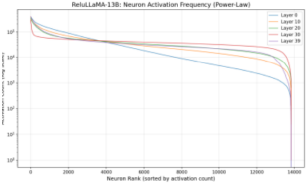
Results: ReluLLaMA-7B speed comparisons

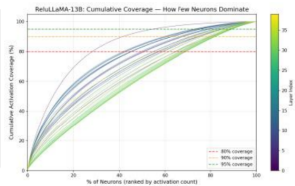


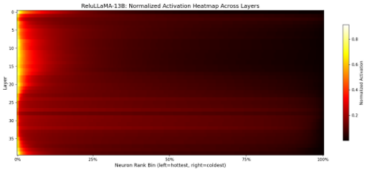
Results: ReluLLaMA-7B activation profiles

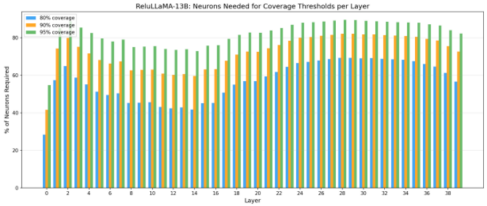


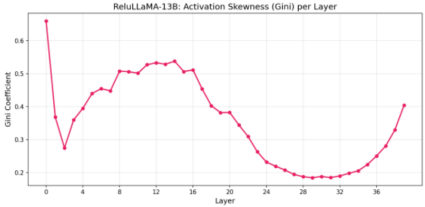
Results: ReluLLaMA-13B











Results: ReluLLaMA-13B

Hot-neuron totals by category

Category	Tok.	Avg hot/layer	% of 13,824
Creative	158	10,789	78.0%
Code	150	10,764	77.9%
Factual	153	10,108	73.1%
Reasoning	159	10,631	76.9%
Conversational	150	10,706	77.4%

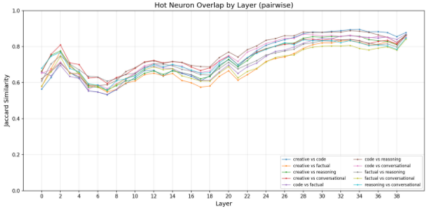
Jaccard similarity of hot-neuron sets

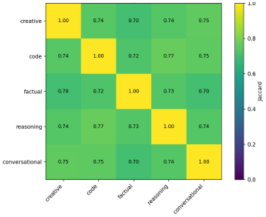
	Creative	*Code*	*Factual*	*Reasoning*	*Convers.*
Creative	1.000	0.741	0.703	0.737	0.754
Code	0.741	1.000	0.725	0.767	0.751
Factual	0.703	0.725	1.000	0.734	0.703
Reasoning	0.737	0.767	0.734	1.000	0.738
Conversational	0.754	0.751	0.703	0.738	1.000

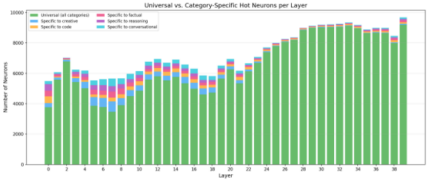
Interpretation

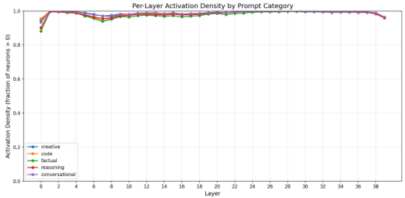
Most prompt categories activate a similar hot-neuron core, but overlap is not perfect. This supports profiling-based placement while leaving room for input-specific prediction.

Results: ReluLLaMA-13B additional analysis









Capabilities, impact, and limitations

What this method enables

- Faster **local assistants**
- Better **privacy** from on-device inference
- Lower **cost** than server-grade GPUs
- More room for **custom local models**

Where it shines most

Best for highly sparse models, small-batch generation, and local low-latency use cases.

Limits to keep in mind

- Smaller gains when sparsity is lower.
- Long prompt-prefill phases reduce the sparsity advantage.
- The system is more complex than plain offloading.

Broader lesson

The win comes from **matching model behavior to hardware hierarchy**. Find the stable hot working set, then keep it near the GPU.

Conclusion

Three things to remember

1. **Observation:** LLM neuron activations are highly skewed, so a small hot set drives most of the work.
2. **Method:** PowerInfer combines offline neuron placement, online activation prediction, neuron-aware sparse operators, and GPU-CPU hybrid execution.
3. **Impact:** It makes large-model local serving much faster on consumer GPUs, with up to **11.69x** speedup and near-A100 performance for some settings.

One-sentence summary

PowerInfer wins because it does not ask the GPU to hold the whole model; it asks the GPU to hold the part of the model that matters most, most of the time.

Questions?

Primary reference: Song et al., "PowerInfer: Fast Large Language Model Serving with a Consumer-grade GPU," SOSP 2024.